2 Grammaires formelles

1 Grammaires syntagmatiques

1.1 Définitions

Nous nous intéressons dans ce cours uniquement à certaines classes de grammaires syntagmatiques. Elle sont fondées sur la notion de vocabulaire terminal (celui du langage associé), de vocabulaire non-terminal (ou auxiliaire) (celui des syntagmes), et de règles de production, avec la façon dont elles sont employées.

Définition 1.1 Une grammaire formelle est un quadruplet $G = (N, T, P, S)$ où :
- N, noté parfois V_N, est le vocabulaire non-terminal
- T, noté parfois V_T, est le vocabulaire terminal
- $N \cap T = \emptyset$ et $N \cup T = V$, vocabulaire « général », est fini (donc N et T aussi).
- $S \in N$ est l'axiome, ou le symbole initial.
- P est un ensemble fini de productions, ou règles, de forme :
  $\alpha \to \beta$, avec $\alpha \in V^+$, $\beta \in V^*$. α est la partie gauche, β la partie droite.
- G est « syntagmatique » si $\alpha \in V^*NV^*$ pour toute règle $\alpha \to \beta$ de P.

Notations :

On notera souvent	a, b, c...	des symboles de T
	A, B, ...	des symboles de N
	U, V, W...	des symboles sur $V = N \cup T$
	u, v, w...	des chaînes sur T (élément de T^*)
	$\alpha, \beta, \gamma \ldots$	des chaînes sur V (éléments de V^*)

Définition 1.2 Une chaîne $\alpha \in V^*$ **dérive** directement vers $\beta \in V^*$ dans G, ce qu'on note $\alpha \vdash_G \beta$, ssi :

$$(\exists\ \theta, \eta \in V^*)(\exists\ \gamma \to \delta \in P)[\alpha = \theta\gamma\eta \ \ \& \ \ \beta = \theta\delta\eta]$$

On dit que α **dérive au sens large** vers β dans G, et on note $\alpha \vdash^*_G \beta$, ssi il existe une suite $\alpha_0, \alpha_1, \ldots, \alpha_m$ de chaînes sur V telle que :

$$\alpha_0 = \alpha \ \ \& \ \ \alpha_m = \beta \ \ \& \ \ (\forall i \ \in [0, m-1])[\alpha_i \underset{G}{\vdash} \alpha_{i+1}]$$

Une dérivation directe consiste donc à remplacer une occurrence γ d'une partie gauche de règle, dans une chaîne α, par sa partie droite δ. Il s'agit donc d'une relation, et pas d'une fonction.
La longueur d'une dérivation (au sens large) est m. Remarquons qu'on a toujours $\alpha \vdash^*_G \alpha (m = 0)$. Une dérivation directe est évidemment une dérivation au sens large de longueur 1.
Si nécessaire, on peut préciser la longueur d'une dérivation en notant $\alpha \vdash^n_G \beta$ (α dérive en β en n dérivations directes).

Définition 1.3 Soit $A \in N$ et $G = (N, T, P, S)$. Les **formes sententielles** de A dans G sont toutes les chaînes dérivables à partir de A dans G :

$$S(G, A) = \{\sigma | A \underset{G}{\vdash^*} \sigma\} =< \{A\}, \{ \underset{G}{\vdash} \} >$$

Les **formes sententielles** de G sont celles de l'axiome S :

$$S(G) = S(G, S) = \{\sigma | S \underset{G}{\vdash^*} \sigma\} =< \{S\}, \{ \underset{G}{\vdash} \} >$$

La notation $< B, R >$ désigne la fermeture inductive d'un ensemble de base B par un ensemble de relations ou règles R. Les formes sententielles peuvent donc contenir des symboles terminaux et non-terminaux.

Définition 1.4 Le langage **engendré par** (ou **associé à**) A dans G est formé des formes sententielles terminales de A dans G :

$$L(G,A) = \{w | w \in T^* \ \ \& \ \ A \overset{*}{\underset{G}{\vdash}} w\} = S(G,A) \cap T^*$$

Le **langage engendré par** (ou **associé à**) $G = (N,T,P,S)$ est le langage associé à l'axiome S :

$$L(G) = L(G,S) = \{w | w \in T^* \ \ \& \ \ S \overset{*}{\underset{G}{\vdash}} w\} = S(G) \cap T^*$$

On notera aussi L_G pour $L(G)$.

Le langage engendré par G est donc l'ensemble des mots formés sur le vocabulaire terminal et dérivables à partir de l'axiome.

Définition 1.5 Deux grammaires $G1$ et $G2$ sont dites (faiblement) équivalentes ssi $L_{G1} = L_{G2}$, ce qu'on note $G1 \sim G2$.

L'équivalence forte sera définie uniquement pour les grammaires associant des arbres syntaxiques aux chaînes du langage associé.

1.2 Exemples commentés

Exemple E2.4:

Soit $G1$ donné par :

$P1 = \{S \to 0S1, S \to 01\}$, avec $T = \{0,1\}$ et $N = \{S\}$
$L(G1) = \{0^n 1^n | n \geq 1\}$

Soit $G2$ donné par :

$P2 = \{S \to 0A1, 0A \to 00A1, A \to \varepsilon\}$, avec $T = \{0,1\}$ et $N = \{S,A\}$
On a encore $L(G2) = \{0^n 1^n | n \geq 1\}$

$G1$ et $G2$ sont donc faiblement équivalentes.
Pour prouver que le langage engendré est bien celui qui est mentionné, il est nécessaire de regarder tout ce qui « peut être engendré » par S, puis de s'intéresser aux chaînes terminales.
Pour l'exemple 1, les formes sententielles sont nécessairement de forme $0^k S 1^k$ ou $0^k 1^k$.
Donc $L(G1) \subseteq \{0^n 1^n | n \geq 1\}$. Réciproquement, pour n donné, $0^n 1^n$ est engendré par la dérivation $S \overset{R1^{n-1}R2}{\vdash} 0^n 1^n$ si on appelle $R1$ la première règle et $R2$ la seconde.
Pour l'exemple 2, on voit qu'il faut appliquer $R1$, qui ne pourra plus être appliquée. Ensuite : toute dérivation commence par $S \vdash 0A1$. Il est immédiat de montrer que toutes les formes sententielles (différentes de S) sont de forme $0^k A 1^k$ ou $0^k 1^k$, et on raisonne alors comme dans l'exemple 1.

Exemple E2.5:

E → E+T | T
T → T*F | F
F → (E) | a | b
G3 engendre l'ensemble des expressions arithmétiques sur a et b (sans le $-$ ni le /).

Par exemple, a*(b+a*b) est engendré par la dérivation :
E ⊢ T ⊢ T*F ⊢ a*F ⊢ a*(E) ⊢ a*(E+T) ⊢ a*(T+T) ⊢ a*(F+T)
⊢ a*(b+T) ⊢ a*(b+T*F) ⊢ a*(b+F*F) ⊢ a*(b+a*F) ⊢ a*(b+a*b)
Cette dérivation est dite « gauche », car, à chaque pas, on a réécrit le non-terminal le plus à gauche de la forme sententielle courante.

1.3 Exercices

Exercice E2.6:

R1 : S → CD R6 : Aa → aA R10 : C → ε
R2 : C → aCA R7 : Ab → bA R11 : D → ε
R3 : C → bCB R8 : Ba → aB
R4 : AD → aD R9 : Bb → bB
R5 : BD → bD

Soit $Ld = \{ww | w \in \{a, b\}^*\}$, le langage des mots « doublés » sur $\{a, b\}$. Montrer que $L_G = Ld$.

Exercice E2.7:

R1 : S → aSBC R4 : aB → ab R6 : bC → bc
R2 : S → aBC R5 : bB → bb R7 : cC → cc
R3 : CB → BC

Soit $L = \{a^n b^n c^n | n \geq 1\}$. Montrer que $L_G = L$.

2 Pourquoi classer les grammaires ?

La réponse est double. D'une part, les langages engendrables par les différents types de grammaires sont différents. D'autre part, les modèles de calcul utilisables pour analyser un langage défini par une grammaire sont différents selon le type de ladite grammaire. La première classification, due au linguiste américain Chomsky et au mathématicien français Schutzenberger, a été proposée dans les années cinquante.

2.1 Définition des types « stricts »

Nous donnons ici une explication succinte de la « hiérarchie de Chomsky ».
Une grammaire G=(N, T, P, S) est dite :

de type 0 si la seule contrainte imposée sur la forme d'une règle $\alpha \to \beta$, est :
$\alpha \in V^* N V^*$ et $\beta \in V^*$.

de type 1 si de plus toutes les règles sont non-raccourcissantes :
$\alpha \to \beta \in P \Rightarrow |\alpha| \leq |\beta|$.

de type 2 si de plus toutes les règles consistent à réécrire un symbole non-terminal unique : $\alpha = A \in N$.
Les règles sont donc de forme :
$A \to \beta$, avec $A \in N$ et $\beta \in V^+$.
Les règles de ce type sont aussi dites hors-contexte strictes, et G est également dite hors-contexte stricte.

de type 3 si de plus toutes les règles sont de forme :
$A \to bC$ ou $A \to b$ avec $A, C \in N$ et $b \in T$.
Les règles de ce type sont aussi dites linéaires à droite strictes, et G est également dite linéaire à droite stricte.

Le lecteur attentif aura remarqué le problème posé par cette classification : il est impossible d'écrire une grammaire de type 1, 2 ou 3 engendrant le mot vide ε. Or on en a tout le temps besoin pour spécifier des langages de programmation, une liste d'objets (par exemple, une liste de déclarations de variables, ou de types) pouvant être vide.
Cela est possible en autorisant des règles de la forme $A \to \varepsilon$, ce qui pose un problème pour le type 1, car on se retrouve dans le type 0, et rend la conception d'algorithmes d'analyse une peu plus délicate pour les types 2 et 3 ainsi « **généralisés** ».
On définit alors les types « **étendus** », en permettant la seule règle d'effacement, $S \to \varepsilon$, si S n'apparaît dans aucune partie droite, et tout va bien pour les types 2 et 3, car on peut transformer une grammaire de type 2 ou 3 généralisée en une grammaire équivalente de type 2 ou 3 « étendu » en « éliminant les epsilons ».

2.2 Dénomination des langages associés

1. On peut montrer que les langages de type 0 sont exactement les langages (sur T) **récursivement énumérables**. Ce sont aussi les langages **semi-décidables**.
   Enfin, ce sont les langages (semi-)reconnaissables ou engendrables par **machines de Turing (MT)**.
2. On peut montrer que, pour toute grammaire de type 1, il en existe une autre, équivalente, elle aussi

de type 1, dont toutes les règles sont de la forme :

$$\gamma A \delta \to \gamma \beta \delta, \quad \gamma, \beta, \delta \in V^*, \quad |\beta| \geq 1, \quad A \in N$$

γ et δ sont appelés les **contextes** gauche et droit de la règle.
Pour cette raison, les langages de type 1 sont dits **sous-contexte** (context sensitive).
Les langages de type 1 sont **décidables**, avec inclusion stricte.
Enfin, ce sont les langages reconnaissables par **automates linéaires bornés** (linear bounded automata, ou MT telles que la reconnaissance d'une chaîne de longueur n se fasse en espace $O(n)$).

3. Les langages de type 2 sont dits **hors contexte** (context free). On dit « hors-contexte » pour « hors-contexte généralisé ».
   Enfin, les langages de type 2 sont exactement les langages reconnaissables par **automates à pile**.
4. Les langages de type 3 sont **linéaires à droite stricts**. On définit par analogie les langages **linéaires à gauche stricts**, et les langages **linéaires à droite (ou à gauche) étendus**.
   Une grammaire est **linéaire à droite généralisée** si toutes ses règles sont de la forme :

$$A \to uB \quad \text{ou} \quad A \to u, \quad \text{avec} \quad A, B \in N \quad \text{et} \quad u \in T^*$$

5. On peut montrer que :
   Tout langage linéaire généralisé est aussi un langage linéaire étendu.
   Tout langage linéaire à droite est aussi **linéaire à gauche**, et inversement.

Il s'agit encore de hiérarchies de langages : si G est une grammaire linéaire (stricte, étendue, généralisée), elle est hors-contexte (stricte, étendue, généralisée), et si G est une grammaire hors-contexte (stricte, étendue, généralisée), elle est sous-contexte (stricte ou étendue). Voir le tableau récapitulatif des types de grammaires formelles (§2.4) pour une présentation plus détaillée.
ATTENTION : Il est parfaitement possible de décrire un langage hors-contexte avec une grammaire de type 1 (non hors contexte), voire avec une grammaire de type 0 (non sous-contexte), mais pas avec une grammaire de type 3.

2.3 Exercices

Exercice E2.8:

R1 : S → aSBC R4 : aB → ab R6 : bC → bc
R2 : S → aBC R5 : bB → bb R7 : cC → cc
R3 : CB → BC

Quel est le type de cette grammaire ?

Exercice E2.9:

R1 : S → CD R6 : Aa → aA R10 : C → ε
R2 : C → aCA R7 : Ab → bA R11 : D → ε
R3 : C → bCB R8 : Ba → aB
R4 : AD → aD R9 : Bb → bB
R5 : BD → bD

Quel est le type de cette grammaire ?

Exercice E2.10:

R1 : S → CD R8 : AD → aD R15 : Bb → bB
R2 : S → EF R9 : BD → bD R16 : C → a
R3 : S → ε R10 : AF → aF R17 : D → a
R4 : C → aCA R11 : BF → bF R18 : E → b
R5 : C → bCB R12 : Aa → aA R19 : F → b
R6 : E → aEA R13 : Ab → bA
R7 : E → bEB R14 : Ba → aB

Quel est le type de cette grammaire ?
Prouver que cette grammaire est équivalente à la précédente (engendre le même langage).

2.4 Tableau récapitulatif des types de grammaires formelles

(0) Grammaire **formelle** : $\alpha_i \in V^+$, $\beta_i \in V^*$
$G = (N, T, P, S) \quad N \cap T = \emptyset \quad N \cup T = V \quad P = \{\alpha_i \to \beta_i\}$

(1) Grammaire **syntagmatique** : $\alpha \to \beta$ avec $\alpha \in V^*NV^*, \beta \in V^*$

(2) Grammaire **de type 0** : $\alpha \to \beta$ avec $\alpha \in N^+, \beta \in V^*$ — $\mathcal{L}(1) = \mathcal{L}(2)$

(3) Grammaire **sous contexte avec effacement généralisé** :
$\gamma A \delta \to \gamma \beta \delta$ avec $A \in N, \gamma\beta\delta \in V^*$ — $\mathcal{L}(2) = \mathcal{L}(3)$

(4) Grammaire **de type 1 strict** (sans effacement) :
$\alpha \to \beta$ avec $\alpha \in V^*NV^*, \beta \in V^*$ et $|\alpha| \leq |\beta|$

(5) Grammaire **sous contexte stricte** :
$\gamma A \delta \to \gamma \beta \delta$ avec $\alpha \in N, \gamma\delta \in V^*, \beta \in V^+$ — $\mathcal{L}(4) = \mathcal{L}(5)$

(6) Grammaire **de type 1 étendu** :
(a) $\alpha \to \beta$ avec $\alpha \in V^*NV^*, \beta \in V^+$ et $|\alpha| \leq |\beta|$
ou (b) $S \to \varepsilon$ et S n'est dans aucune partie droite de règle.

(7) Grammaire **sous contexte étendue** : — $\mathcal{L}(6) = \mathcal{L}(7)$
(a) $\gamma A \delta \to \gamma \beta \delta$ avec $A \in N, \gamma\delta \in V^*, \beta \in V^+$
ou (b) $S \to \varepsilon$ et S n'est dans aucune partie droite de règle.
L'extension permet d'ajouter ε aux langages stricts, sans modifier les théorèmes.

(8) Grammaire **de type 2 strict** (ou **hors-contexte stricte**) :
$A \to \alpha$ avec $A \in N, \alpha \in V^+$

(9) Grammaire **de type 2 étendu** (ou **hors-contexte étendu**) :
(a) $A \to \alpha$ avec $A \in N, \alpha \in V^+$
ou (b) $S \to \varepsilon$ et S n'est dans aucune partie droite de règle.

(10) Grammaire **de type 2 généralisé** (ou **hors-contexte** (généralisée)) :
$A \to \alpha$ avec $A \in N, \alpha \in V^*$ — $\mathcal{L}(9) = \mathcal{L}(10)$

(11) Grammaire **de type 3 strict** (ou **linéaire droite stricte**) :
(a) $A \to aB$ avec $A, B \in N, a \in T$
ou (b) $A \to a$

(12) Grammaire **de type 3 étendu** (ou **linéaire droite étendue**) :
(a) $A \to aB$ avec $A, B \in N, a \in T$
ou (b) $A \to a$
ou (c) $S \to \varepsilon$ et S n'est dans aucune partie droite de règle.

(13) Grammaire **de type 3 généralisé** (ou **linéaire droite** (généralisée) :
(a) $A \to wB$ avec $A, B \in N, w \in T^*$
ou (b) $A \to w$ — $\mathcal{L}(12) = \mathcal{L}(13)$

3 Connaître le type d'un langage

Comme nous avons pu le voir dans les sections précédentes, on peut connaître facilement le type d'une grammaire. Ainsi, si on dispose d'une grammaire, on peut savoir quelle vont être les difficultés pour l'implémenter. Par contre, on ne sait pas si le langage engendré ne pourrait pas être plus efficacement implémenté par une grammaire d'un type supérieur.
Pour cela, il faut être capable de déterminer la classe d'un langage, indépendamment de toute grammaire. C'est le but des deux lemmes qui sont exposés ici, le lemme de l'étoile et le théorème des paires itérantes (aussi nommé théorème des gendarmes ou lemme de Bar-Hillel).

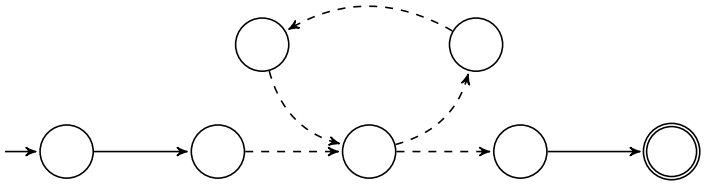
3.1 Lemme de l'étoile

Lemme 3.1 (Lemme de l'étoile)
Pour tout langage **rationnel** L :
$\exists k \in \mathbb{N}$ tel que $\forall x \in L$,
$|x| \geq k \Rightarrow [\exists u, v, w \in V^*$ avec $|v| > 0$ et $|uv| \leq k$ tels que $x = uvw$ et $\forall i\ (uv^i w \in L)]$ [a]

a. Il existe plusieurs variantes dont une très répandue ayant la condition $|v| \leq k$ au lieu de $|uv| \leq k$. La variante donnée ici est plus "forte" et permet des démonstration en général plus simples.

Ce lemme signifie que pour tout langage rationnel, si l'on considère un mot assez grand ($\geq k$), on peut répéter indéfiniment au moins l'un de ses infixes, tout en restant dans le langage L.

Nous ne donnerons pas la démonstration de ce théorème, mais on peut comprendre assez facilement la raison de ce phénomène.
Comme nous le savons déjà, un langage rationnel peut être analysé par un automate d'états finis (déterministe). Si le mot est assez grand (s'il a plus de caractères qu'il n'y a d'états dans l'automate), il ne peut être reconnu dans l'automate qu'en passant 2 fois (au moins) par un même état :



Ainsi, les mots obtenus en passant 2, 3, ... i fois par la boucle ainsi formée font tous partie du langage L.
Ce lemme peut être utilisé pour des démonstrations par l'absurde. On ne s'en servira par exemple que pour prouver qu'un langage n'est **pas rationnel**.

3.2 Théorème des paires itérantes

Théorème 3.1 (Théorème des paires itérantes)
Pour tout langage **hors contexte** L :
$\exists p \in \mathbb{N}$ tel que $\forall u \in L$ si $|u| \geq p$, alors on a :
$u = vwxyz$ avec $wy \neq \varepsilon$ et $|wxy| \leq p$ et
$\forall k \geq 0, u_k = vw^k xy^k z \in L$

Ce théorème ressemble un peu au lemme de l'étoile.
Nous ne donnerons pas non plus sa démonstration, mais il est aussi assez aisé de comprendre les raisons de ce phénomène.

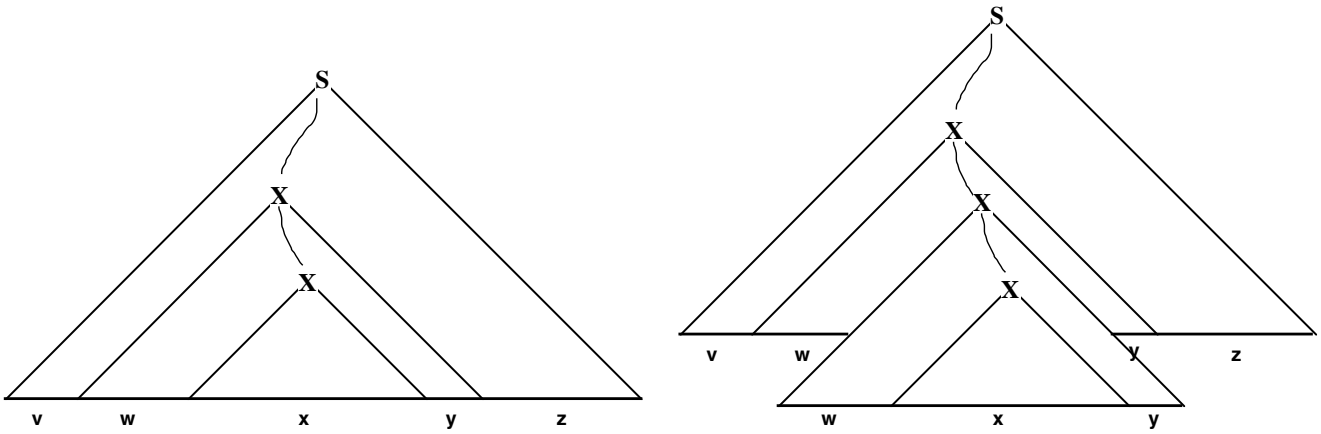


FIGURE 2.1 – Illustration du principe de preuve du théorème des paires itérantes

Si un langage est hors contexte, on peut construire un arbre de dérivation de chacun de ses mots. Si un mot est suffisamment long, on trouve un arbre où l'un des non-terminaux est présent récursivement dans l'ensemble de ses descendants (cf. figure 2.1). Ainsi, les mots construits en itérant autant de fois qu'on le souhaite la récursion ainsi détectée font eux aussi partie du langage (cf. figure 2.1).
Ce lemme est aussi utilisé pour des démonstrations par l'absurde. On s'en servira, par exemple, pour prouver qu'un langage n'est **pas hors contexte**.

3.3 Exercices

Exercice E2.11:

Le langage $L = \{w | w = a^n b^n, n \geq 0\}$ est-il rationnel ?

Exercice E2.12:

Le langage $L = \{w | w = a^n b^n c^n, n \geq 0\}$ est-il hors contexte ?

Exercice E2.13:

Même question avec le langage $L = \{ww | w \in (a+b)^*\}$.

4 Grammaire de type 1 et grammaires sous contexte

4.1 Introduction

Lorsque nous avons expliqué la classification des grammaires (cf. 2.2), nous avons indiqué que le grammaires de type 1 permettent d'engendrer les langages nommés langages sous-contexte.
La raison pour laquelle ces langages sont ainsi nommés est qu'il existe un théorème disant que :

Théorème 4.1 Toute grammaire de type 1 est équivalente à une grammaire (de type 1) dont les règles satisfont à la contrainte plus précise :

$$\gamma A \delta \to \gamma \alpha \delta, \quad \gamma, \alpha, \delta \in V^*, \quad |\alpha| \geq 1, \quad A \in N$$

On pourra aussi noter :

$$A \to \alpha / \gamma _ \delta, \text{ où } \delta \text{ est nommé contexte gauche et } \delta \text{ contexte droit}$$

L'intérêt de cette transformation est évident : si les règles se contentent de transformer un non-terminal en une chaîne, il sera possible de construire un arbre de dérivation pour cette grammaire.
Nous allons voir comment cette transformation peut être faite.

4.2 Algorithme de transformation

Étape 1 si $x \in V_T$ apparaît en partie gauche d'une règle, on crée un nouveau symbole $<X>$ dans V_N.
On réécrit ensuite l'ensemble des règles en conséquence.
ex : $xAy \to \alpha$ est réécrit en $<X>A<Y> \to \alpha$. On fait de même sur les parties droites de règles puis on rajoute les règle $<X> \to x$ dans P.
On obtient ainsi une nouvelle grammaire n'ayant aucun terminal en partie gauche de règle.

Étape 2 : chaque règle de la forme $X_1 \dots X_k \to Y_1 Y_2 \dots Y_m$ (non encore sous-contexte) est remplacée par l'ensemble de règles :
$X_1 \to X'_1 / _ X_2 \dots X_k$
$X_2 \to X'_2 / X'_1 _ X_3 \dots X_k$
$\dots$
$X_k \to Y_k \dots Y_m / X'_1 \dots X'_{k-1} _$
$X'_{k-1} \to Y_{k-1} / _ Y_k$
$\dots$
$X'_1 \to Y_1 / _ Y_2$
et on ajoute les nouveaux non-terminaux X'_i créés. On a ainsi une grammaire sous-contexte.

4.3 Exercices

Exercice E2.14:

Faire le passage en sous-contexte des grammaires de type 1 vues dans les documents précédents. Générer l'un des mot du langage associé et dessinez l'arbre de dérivation correspondant.